
SISTEMA DE FACTURACIÓN Y PAGOS – INDUSTRIA TÍPICA GUATEMALTECA, S.A

202308487 – Zabdi Merari Adina Donis Rosales

Resumen

El presente trabajo describe el diseño, desarrollo e implementación del Proyecto No. 3 del curso de Introducción a la Programación y Computación 2. Se desarrolló un sistema integral de facturación y pagos para la empresa Industria Típica Guatemalteca, S.A., compuesto por una API RESTful implementada en C# con ASP.NET Core y un Frontend desarrollado en React. La comunicación entre ambos programas se realiza mediante el protocolo HTTP. Los datos se persisten en archivos XML, garantizando la continuidad de la información entre sesiones. El sistema permite cargar configuraciones de clientes y bancos, procesar facturas y pagos, aplicar abonos a facturas pendientes ordenadas por antigüedad, gestionar saldos a favor, consultar estados de cuenta y visualizar gráficas de ingresos por banco para los últimos tres meses.

Palabras clave

API RESTful, ASP.NET Core, React, XML, programación orientada a objetos.

Abstract

This paper describes the design, development and implementation of Project No. 3 for the Introduction to Programming and Computing 2 course. A comprehensive billing and payment system was developed for the company Industria Típica Guatemalteca, S.A., composed of a RESTful API implemented in C# with ASP.NET Core and a Frontend developed in React. Communication between both programs is carried out using the HTTP protocol. Data is persisted in XML files, ensuring information continuity between sessions. The system allows loading client and bank configurations, processing invoices and payments, applying credits to pending invoices ordered by date, managing credit balances, querying account statements, and displaying income charts by bank for the last three months.

Keywords

RESTful API, ASP.NET Core, React, XML, object-oriented programming.

Introducción

El desarrollo de sistemas de software orientados a objetos que utilizan protocolos estándar de comunicación como HTTP es una habilidad fundamental en la ingeniería de sistemas. Este proyecto tiene como objetivo construir una solución integral que implemente una API RESTful para gestionar los procesos de facturación y pagos de una empresa, aplicando los principios de la Programación Orientada a Objetos y el uso de archivos XML como mecanismo de persistencia de datos.

ITGSA ha creado un canal de distribución en línea llamado Tienda Virtual, mediante el cual los clientes adquieren productos y servicios. Los pagos pueden realizarse a través de la banca en línea de distintos bancos, y el sistema debe aplicarlos de forma automática a las facturas pendientes más antiguas de cada cliente. El presente ensayo describe la arquitectura implementada, la lógica de negocio desarrollada y las decisiones técnicas tomadas durante el proceso.

Desarrollo del tema

a. Arquitectura del sistema.

El sistema sigue una arquitectura cliente-servidor en dos programas. El Backend es una API RESTful en C# con ASP.NET Core que expone endpoints HTTP. El Frontend es una aplicación React que consume dichos endpoints mediante Axios. La arquitectura interna del Backend se divide en cuatro capas con responsabilidades claramente separadas: Models, Repositories, Services y Controllers.

Los Models definen la estructura de las entidades del sistema: Cliente, Banco, Factura y Pago. El Repository gestiona el acceso y la persistencia de datos en archivos XML, registrado como Singleton para mantener una única instancia en memoria durante toda la ejecución del servidor. Los Services contienen la lógica de negocio y se registran como Scoped, creando una nueva instancia por cada

petición HTTP. Los Controllers exponen los endpoints y delegan el procesamiento a los servicios correspondientes.

b. Archivos de entrada y procesamiento

El sistema acepta dos tipos de archivos XML de entrada. El archivo config.xml contiene la configuración de clientes y bancos. Al procesarlo, el sistema verifica si cada NIT de cliente o código de banco ya existe; si existe, actualiza la información, y si no, crea un nuevo registro. La respuesta XML indica la cantidad de registros creados y actualizados.

El archivo transac.xml contiene facturas generadas por la Tienda Virtual y pagos realizados desde las bancas virtuales. Para las facturas se valida que el cliente exista, que la fecha tenga el formato correcto y que el número de factura no esté duplicado. Para los pagos se valida que el banco y el cliente existan y que la fecha sea válida.

c. Lógica de abono a facturas

La lógica de aplicación de pagos es el núcleo del sistema. Al recibir un pago, el sistema suma el monto recibido al saldo a favor existente del cliente para obtener un remanente total disponible. Este remanente se aplica a las facturas pendientes del cliente, ordenadas de más antigua a más reciente. Si el remanente supera el saldo pendiente de una factura, esta queda completamente pagada y el excedente continúa hacia la siguiente. Si al finalizar el procesamiento queda remanente, este se guarda como saldo a favor del cliente para aplicarse automáticamente en futuras compras.

Del mismo modo, cuando se registra una factura nueva y el cliente tiene saldo a favor, dicho saldo se aplica inmediatamente a la factura antes de guardarla, reduciendo el saldo pendiente desde el momento del registro.

d. Persistencia de datos en XML

Los datos se persisten en cuatro archivos XML: clientes.xml, bancos.xml, facturas.xml y pagos.xml, ubicados en la carpeta Data del Backend. Al iniciar el servidor, el repositorio carga automáticamente los datos existentes en las listas en memoria. Después de cada operación de escritura, los datos se guardan inmediatamente en los archivos XML correspondientes, garantizando que la información persista entre reinicios del servidor.

Esta solución cumple con el requisito de base de datos del proyecto sin necesidad de un motor de base de datos externo, aprovechando las capacidades de la librería System.Xml.Linq de .NET para la lectura y escritura de XML de forma programática.

Tabla I.
Endpoints de la API RESTful implementada

ENDPOINT	DESCRIPCIÓN
POST /api/Config/cargar VARIABLE	Recibe config.xml y registra o actualiza clientes y bancos.
POST /api/Transaccion/cargar	Recibe transac.xml y procesa facturas y pagos.
GET /api/Consulta/clientes	Devuelve todos los clientes ordenados por NIT.
GET /api/Consulta/cliente/{nit}	Devuelve el estado de cuenta de un cliente específico.
GET /api/Consulta/ingresos- bancos	Devuelve ingresos agrupados por banco de los últimos 3 meses.
POST /api/Reset/limpiar	Elimina todos los datos y reinicia los archivos XML.

Fuente: elaboración propia

Conclusiones

- Se implementó exitosamente una API RESTful en C# con ASP.NET Core que gestiona los procesos de facturación y pagos de ITGSA, aplicando correctamente el paradigma de programación orientada a objetos mediante la separación en capas de Models, Repositories, Services y Controllers.
- La lógica de aplicación de pagos a facturas funciona conforme al enunciado, priorizando siempre la factura más antigua del cliente y gestionando el saldo a favor cuando el pago supera el total de deudas pendientes.
- La persistencia de datos mediante archivos XML garantiza que la información no se pierde entre reinicios del servidor, cumpliendo el requisito de base de datos sin necesidad de un motor externo.
- El Frontend en React consume correctamente todos los endpoints y presenta la información de forma clara, incluyendo la exportación de estados de cuenta y gráficas a formato PDF mediante el

diálogo de impresión del navegador.

- El uso de Swagger durante el desarrollo facilitó la verificación de cada endpoint de forma independiente al Frontend, agilizando el proceso de detección y corrección de errores.

Referencias bibliográficas

Axios Contributors. (2024). Axios HTTP client documentation. <https://axios-http.com/docs/intro>

Chart.js Contributors. (2024). Chart.js documentation. <https://www.chartjs.org/docs/>

Microsoft Corporation. (2024). ASP.NET Core Web API documentation. <https://learn.microsoft.com/en-us/aspnet/core/>

Meta Platforms, Inc. (2024). React – A JavaScript library for building user interfaces. <https://react.dev/>

Tailwind Labs. (2024). Tailwind CSS documentation. <https://tailwindcss.com/docs>

Apéndices

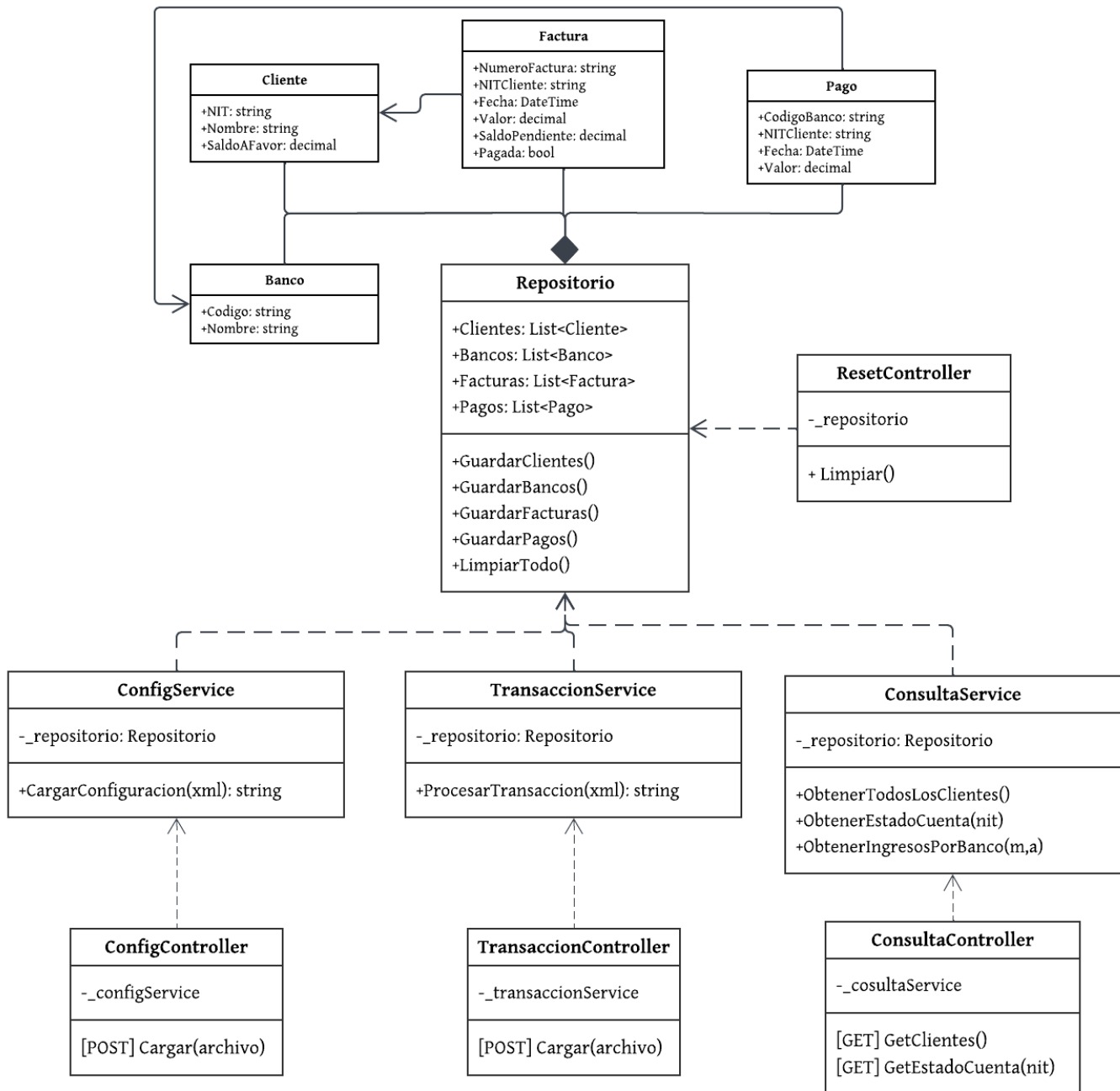


Figura 1. Diagrama de Clases.

Fuente: elaboración propia.

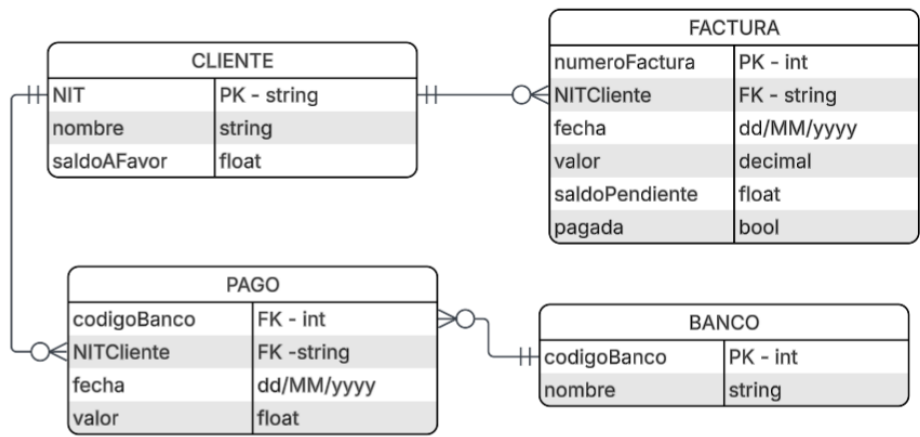


Figura 2. Diagrama de Entidad-Relación.

Fuente: elaboración propia.